Chapter 6 – Data Modeling I

IERG4320/IEMS5726 Data Science in Practice | Cheat Sheet

Machine Learning Basics

ML = 让计算机从数据中学习规律，而不是人工写规则

ML Types

- **Supervised**: labeled data → Regression / Classification 有标签
- **Unsupervised**: no labels → Clustering / Dim. Reduction 无标签，找规律
- **Semi-supervised**: small labeled + large unlabeled 少量标签+大量无标签
- **Reinforcement**: agent learns via rewards in environment 强化学习，通过奖励学

Regression vs Classification

Regression: predict continuous value (price, count)

回归：预测连续值，如房价

Classification: predict discrete class (spam/not, cat/dog)

分类：预测离散类别，如垃圾邮件检测

Classification Algorithms

1. Decision Tree

决策树：用属性做判断，像if-else树状结构

Key: uses Gini Index or Entropy to split nodes.
Lower Gini = better split (less misclassification).

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics
# Load data
X = pima[feature_cols]  # Features
y = pima.Outcome        # Target
X_train, X_test, y_train, y_test = \
  train_test_split(X, y, test_size=0.3,
                   random_state=1)
# Build and evaluate
model = DecisionTreeClassifier()
model = model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:",
      metrics.accuracy_score(y_test, y_pred))
```

2. Random Forest

随机森林：多棵决策树投票，减少过拟合

Average multiple decision trees trained on different data subsets. Final output = majority vote.

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(
  n_estimators=50, max_features="sqrt",
  random_state=44)
rf.fit(X_train, y_train)
preds = rf.predict(X_test)
print(metrics.accuracy_score(y_test, preds))
```

n_estimators=树的数量，max_features=每次分裂用的特征数

3. SVM (Support Vector Machine)

SVM：找最大间隔的超平面来分类

Find the hyperplane with maximum margin. Support vectors = data points closest to boundary.

Kernels: linear, poly, rbf (for non-linear data).

```
from sklearn import svm, datasets, metrics
cancer = datasets.load_breast_cancer()
X_train, X_test, y_train, y_test = \
  train_test_split(cancer.data,cancer.target,
                   test_size=0.3,random_state=109)
clf = svm.SVC(kernel='linear')
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Accuracy:",
      metrics.accuracy_score(y_test, y_pred))
# Accuracy: 0.9649
```

4. KNN (K-Nearest Neighbor)

KNN：找最近的K个邻居，多数投票决定类别

Classify based on majority class of K nearest neighbors. Distance: Euclidean.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
irisData = load_iris()
X_train,X_test,y_train,y_test = \
  train_test_split(irisData.data,irisData.target,
                   test_size=0.2,random_state=42)
knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(X_train, y_train)
print("Accuracy:",
      metrics.accuracy_score(y_test,
                             knn.predict(X_test)))
# Accuracy: 0.9667
```

5. ANN (Artificial Neural Network)

人工神经网络：模拟神经元连接，可建模非线性关系

```
import torch
import torch.nn as nn
n_input, n_hidden, n_out = 10, 15, 1
batch_size, learning_rate = 100, 0.01
data_x = torch.randn(batch_size, n_input)
data_y = (torch.rand(size=(batch_size,1))<0.5)\
         .float()
# Define model
model = nn.Sequential(
  nn.Linear(n_input, n_hidden), nn.ReLU(),
  nn.Linear(n_hidden, n_out), nn.Sigmoid())
loss_function = nn.MSELoss()
optimizer = torch.optim.SGD(
  model.parameters(), lr=learning_rate)
# Training loop
losses = []
for epoch in range(50000):
  pred_y = model(data_x)
  loss = loss_function(pred_y, data_y)
  losses.append(loss.item())
  model.zero_grad()
  loss.backward()   # Backpropagation
  optimizer.step()  # Gradient descent
```

ReLU隐藏层 + Sigmoid输出层(二分类); MSELoss + SGD优化

Classification Metrics

评估指标：衡量模型预测好不好

Accuracy

$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{total predictions}}$$

准确率 = 预测对的/总数。缺点：类别不平衡时不可靠

```
from sklearn import metrics
metrics.accuracy_score(y_test, y_pred)
```

Confusion Matrix

混淆矩阵：TP/TN/FP/FN四个值

	Pred Pos	Pred Neg
Actual Pos	TP 真阳	FN 漏报
Actual Neg	FP 误报	TN 真阴

```
from sklearn.metrics import confusion_matrix
m = confusion_matrix(y_test, y_pred)
```

Precision & Recall

$\text{Precision} = \frac{TP}{TP+FP}$ 预测为正中，真正为正的比例

$\text{Recall} = \frac{TP}{TP+FN}$ 实际为正中，被正确找出的比例

Trade-off: high threshold → high precision, low recall; low threshold → opposite.

阈值高=保守(精确但漏检多)，阈值低=激进(召回高但误报多)

F1 Score

$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}}$ F1是精确率和召回率的调和平均

Flaw: F1 is class-variant (flipping pos/neg changes F1).

MCC (Matthews Correlation Coefficient)

MCC：对不平衡数据更公平，范围[-1, 1]

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

Advantage: class-invariant, range $[-1, 1]$.

```
metrics.matthews_corrcoef(y_test, y_pred)
metrics.f1_score(y_test, y_pred)
```

AUC-ROC

ROC曲线下面积：衡量模型区分正负样本的能力

ROC = plot TPR vs FPR at different thresholds.

$\text{TPR} = \frac{TP}{TP+FN}$, $\text{FPR} = \frac{FP}{FP+TN}$

AUC = area under ROC curve. AUC=0.5 = random guess, AUC=1 = perfect.

Model Comparison (10-fold CV)

10折交叉验证：公平比较不同算法

```
from sklearn.model_selection import KFold, \
  cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

models = []
models.append(('LR',
  LogisticRegression(solver="liblinear")))
models.append(('KNN',
  KNeighborsClassifier(n_neighbors=5)))
models.append(('SVM',SVC(kernel='rbf')))
models.append(('NB', GaussianNB()))
models.append(('DT',
  DecisionTreeClassifier(criterion="entropy")))
models.append(('RF',
  RandomForestClassifier(n_estimators=100,
                    criterion="entropy")))
for name, model in models:
  kfold = KFold(n_splits=10)
  result = cross_val_score(model, X, y,
    cv=kfold, scoring='accuracy')
  print("%s: Mean=%.2f%% SD=%.2f%%" %
    (name,result.mean()*100,result.std()*100))
```

Hyperparameter Tuning

超参数调优：选最佳模型设置

Parameters (learned): weights, biases, centroids

Hyperparameters (set by user): learning rate, # layers, dropout, epochs, batch size, kernel

Tuning methods:

1. **Grid Search**: try all combinations 网格搜索
2. **Manual**: domain expert picks 专家经验
3. **Random Search**: random combos 随机搜索

不确定就用默认参数，不要只报告最好的一次

Regression

回归：预测连续值，分析变量间关系

Linear Regression

$y = a + bX$ (simple) 一元线性回归

$y = b_0 + b_1x_1 + b_2x_2 + \cdots + b_nx_n$ (multiple)

多元线性回归：多个特征预测一个目标值

Vector form: $y = \theta^T x$

Learning = find best θ via Ordinary Least Squares (OLS).

```
from sklearn import linear_model
import statsmodels.api as sm
X = df[['Interest_Rate','Unemployment_Rate']]
Y = df['Stock_Index_Price']
regr = linear_model.LinearRegression()
regr.fit(X, Y)
print('Intercept:', regr.intercept_)
print('Coef:', regr.coef_)
# Predict
pred = regr.predict([[2.75, 5.3]])
```

R^2 and Adjusted R^2

R^2：模型拟合好坏，越接近1越好

$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$ where $SS_{tot} = \sum(y_i - \bar{y})^2$, $SS_{res} = \sum(y_i - \hat{y}_i)^2$

Problem: R^2 never decreases with more variables.

Adjusted R^2修正了变量数量的影响

$R^2_{adj} = 1 - (1 - R^2)\frac{n-1}{n-k-1}$ (k = # of features)

```
r2 = regr.score(X, Y)   # 0.8976
adj_r2 = 1-(1-r2)*(len(Y)-1)\
          /(len(Y)-X.shape[1]-1)  # 0.8879
```

statsmodels OLS

statsmodels给出完整统计信息：p值、置信区间等

```
X_sm = sm.add_constant(X)  # add intercept
model = sm.OLS(Y, X_sm).fit()
print(model.summary())
# Shows: R2, Adj R2, F-statistic,
#        coef, std err, t, P>|t|, CI
```

Regression Metrics

回归评估：MAE, RMSE, MSE

MAE $= \frac{1}{n}\sum|y_i - \hat{y}_i|$ 平均绝对误差

MSE $= \frac{1}{n}\sum(y_i - \hat{y}_i)^2$ 均方误差，对大偏差惩罚更重

RMSE $= \sqrt{\text{MSE}}$ 均方根误差

```
from sklearn.metrics import mean_squared_error,\
  r2_score, mean_absolute_error
print('MSE:', mean_squared_error(y_test, y_pred))
print('MAE:', mean_absolute_error(y_test, y_pred))
```

Logistic Regression

逻辑回归：用sigmoid把线性输出转成0-1概率，用于分类

Apply sigmoid to linear regression output:

$g(z) = \frac{1}{1+e^{-z}}$ where $z = \theta^T x$

Output bounded in $[0, 1]$; tends to 0 or 1.

```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(random_state=0)
clf.fit(X_train, Y_train)
Y_pred = clf.predict(X_test)
m = confusion_matrix(Y_test, Y_pred)
# ROC curve
from sklearn.metrics import roc_curve, \
  roc_auc_score
lr_probs = clf.predict_proba(X_test)[:, 1]
lr_auc = roc_auc_score(Y_test, lr_probs)
fpr, tpr, _ = roc_curve(Y_test, lr_probs)
```

More Regression Models

Ridge Regression

岭回归：加L2正则化，防止过拟合

Adds L2 penalty. α controls penalty weight.

```
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
pipeline = make_pipeline(
  StandardScaler(), Ridge(alpha=1.0))
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
print('MSE:', mean_squared_error(y_test, y_pred))
print('R2:', r2_score(y_test, y_pred))
```

LASSO Regression

LASSO：加L1正则化，可以做特征选择(系数变0)

```
from sklearn.linear_model import Lasso
model = Lasso(alpha=1.0)
model.fit(X, y)
yhat = model.predict([row])
```

ANN Regression (PyTorch)

神经网络回归：输出层用ReLU(不是Sigmoid)，损失用MSE

```
n_input, n_hidden, n_out = 7, 5, 1
model = nn.Sequential(
  nn.Linear(n_input, n_hidden), nn.ReLU(),
  nn.Linear(n_hidden, n_out), nn.ReLU())
loss_function = nn.MSELoss()
optimizer = torch.optim.SGD(
  model.parameters(), lr=0.01)
for epoch in range(5000):
  pred_y = model(data_x)
  loss = loss_function(pred_y, data_y)
  model.zero_grad()
  loss.backward()
  optimizer.step()
```

SVR (Support Vector Regressor)

支持向量回归：用核函数处理非线性回归

```
from sklearn.svm import SVR
sc_X, sc_y = StandardScaler(), StandardScaler()
X = sc_X.fit_transform(X)
y = sc_y.fit_transform(y)
regressor = SVR(kernel='rbf')
regressor.fit(X, y)
y_pred = regressor.predict(X_new)
```

Parameters vs Hyperparameters

参数=模型学出来的; 超参数=人手动设的

Model	Params	Hyperparams
MLR	coef, bias	None
Logistic	coef, bias	solver, penalty
Ridge	coef, bias	λ (alpha)
LASSO	coef, bias, L1	λ (alpha)

Regression Comparison: Preprocess → Split 70/30 → Run MLR, Ridge, LASSO, SVR-rbf, ANN → Compare R^2, MAE, MSE. 回归模型对比流程：和分类类似，用回归指标评估